

Smart Warehouse Management System (SWMS)

Database Project Documentation

Course: Advanced Database

Platform: Microsoft SQL Server

Project Type: Enterprise Warehouse & Logistics Management System

1. Project Information

Project Title: Smart Warehouse Management System (SWMS)

Course: Database Systems

Database Platform: Microsoft SQL Server

Project Type: Enterprise Warehouse & Logistics Management System

2. Introduction

The Smart Warehouse Management System (SWMS) is an enterprise-level warehouse and logistics database system designed to manage warehouse operations efficiently. The system simulates real-world warehouse environments similar to Amazon and Jumia fulfillment centers.

The main objective of the system is to:

- Receive products from suppliers
- Store products inside warehouse locations
- Manage inventory movement
- Process customer orders
- Optimize delivery operations
- Track users and operations
- Improve database performance using indexing and optimization

The project is implemented using Microsoft SQL Server and includes advanced database concepts such as Stored Procedures, Functions, Triggers, Views, Indexes, Query Optimization, Security & Permissions, Audit Logging, and Backup & Restore.

3. Problem Statement

Large warehouse systems handle millions of products and orders daily. Managing such systems manually causes:

- Slow inventory tracking
- Data inconsistency
- Delayed order processing
- Security risks
- Poor delivery optimization
- Lack of auditing
- Poor query performance

The proposed system solves these issues by providing a scalable and optimized warehouse database management system.

4. System Overview

The system contains multiple departments:

4.1 Inbound Department

Responsible for: Receiving shipments, Validating products, Registering inventory, Generating product units.

4.2 Inventory Department

Responsible for: Storing products, Managing warehouse locations, Inventory movement tracking, Capacity management.

4.3 Outbound Department

Responsible for: Picking orders, Packing orders, Shipping products.

4.4 Network Department

Responsible for: Grouping nearby orders, Delivery optimization, Shipment batching.

4.5 Administration Department

Responsible for: Managing users, Permissions, Security, Audit monitoring.

5. Functional Requirements

User Management

- User login/logout
- Role management
- Permission control
- User activity tracking

Supplier Management

- Add supplier
- Update supplier
- Delete supplier
- View suppliers

Product Management

- Add products
- Generate SKU
- Generate UID for each unit
- Product categorization

Warehouse Management

- Add warehouse locations
- Assign products to locations
- Track product movement
- Capacity checking

Order Management

- Create orders
- Add order items
- Picking operations
- Shipping operations

Reporting

- Inventory reports
- Top ordered products
- Customer reports
- Employee reports

Security & Auditing

- Login tracking
- Operation logging
- Restrict unauthorized actions
- Prevent database deletion

6. Non-Functional Requirements

Requirement	Description
Performance	Fast query execution
Scalability	Support 1M+ records
Security	Role-based access control
Reliability	Audit logging
Availability	24/7 support
Maintainability	Modular SQL structure

7. Database Design

Security Tables

- Users
- Roles
- Departments
- Permissions

Warehouse Tables

- Warehouses
- WarehouseLocations
- InventoryMovements

Product Tables

- Products
- ProductUnits

Supplier Tables

- Suppliers
- InboundShipments

Customer Tables

- Customers
- Orders
- OrderItems

Audit Tables

- AuditLogs
- LoginAudit

8. Entity Relationship Diagram (ERD)

Main Relationships:

Table 1	Relationship	Table 2
Roles	1:M	Users
Departments	1:M	Users
Suppliers	1:M	InboundShipments
Products	1:M	ProductUnits
Warehouses	1:M	WarehouseLocations
Customers	1:M	Orders
Orders	1:M	OrderItems
Products	1:M	OrderItems
ProductUnits	1:M	InventoryMovements

9. SQL Server Implementation

Database Creation:

```
CREATE DATABASE SmartWarehouseDB;  
GO
```

```
USE SmartWarehouseDB;  
GO
```

10. Table Creation

Example: Products Table

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY IDENTITY(1,1),  
    SKU VARCHAR(100) UNIQUE NOT NULL,  
    ProductName VARCHAR(255) NOT NULL,  
    Category VARCHAR(100),  
    Weight DECIMAL(10,2),  
    Dimensions VARCHAR(100)  
);
```

Example: Orders Table

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY IDENTITY(1,1),  
    CustomerID INT NOT NULL,  
    OrderDate DATETIME DEFAULT GETDATE(),  
    Status VARCHAR(50) DEFAULT 'Pending',  
  
    FOREIGN KEY (CustomerID)  
    REFERENCES Customers(CustomerID)  
);
```

11. Stored Procedures

Purpose: Stored Procedures are used to automate warehouse operations and improve performance.

Example: Add Supplier

```
CREATE PROCEDURE sp_AddSupplier  
    @CompanyName VARCHAR(255),  
    @ContactName VARCHAR(255),  
    @Phone VARCHAR(20),  
    @Address VARCHAR(255)  
AS  
BEGIN  
    INSERT INTO Suppliers (CompanyName, ContactName, Phone, Address)  
    VALUES (@CompanyName, @ContactName, @Phone, @Address);  
END;
```

Example: Add Product

```
CREATE PROCEDURE sp_AddProduct  
    @SKU VARCHAR(100),
```

```

        @ProductName VARCHAR(255),
        @Category VARCHAR(100),
        @Weight DECIMAL(10,2),
        @Dimensions VARCHAR(100)
    AS
    BEGIN
        INSERT INTO Products (SKU, ProductName, Category, Weight, Dimensions)
        VALUES (@SKU, @ProductName, @Category, @Weight, @Dimensions);
    END;

```

12. SQL Functions

Purpose: Functions are used to calculate reusable business logic.

Example: Get Available Quantity

```

CREATE FUNCTION fn_GetAvailableQuantity (@ProductID INT)
RETURNS INT
AS
BEGIN
    DECLARE @Qty INT;
    SELECT @Qty = COUNT(*)
    FROM ProductUnits
    WHERE ProductID = @ProductID AND Status = 'Available';
    RETURN @Qty;
END;

```

13. Triggers

Purpose: Triggers automatically execute actions after database events.

Audit Trigger:

```

CREATE TRIGGER trg_AuditProducts
ON Products
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    INSERT INTO AuditLogs (Username, Operation, ObjectName, ObjectType,
    OperationDate)
    VALUES (SYSTEM_USER, 'MODIFY', 'Products', 'TABLE', GETDATE());
END;

```

Prevent Database Deletion Trigger:

```

CREATE TRIGGER trg_PreventDatabaseDrop
ON ALL SERVER
FOR DROP_DATABASE
AS
BEGIN

```

```

PRINT 'DROP DATABASE is not allowed';
ROLLBACK;
END;

```

14. Views

Purpose: Views simplify data access and improve security.

Inventory View:

```

CREATE VIEW vw_CurrentInventory AS
SELECT p.ProductName, p.SKU, pu.UID, pu.Status
FROM ProductUnits pu
JOIN Products p ON pu.ProductID = p.ProductID;

```

15. Users & Permissions

Role	Permissions
Admin	Full access
InventoryManager	Inventory management
Receiver	Inbound operations
Picker	Order picking
NetworkManager	Delivery optimization

Example Permission:

```

GRANT SELECT, INSERT, UPDATE ON Products TO InventoryManagerUser;

```

16. Audit & Logging System

Login Auditing records: Username, Login time, Operations performed, Object name, Object type.

Login Trigger:

```

CREATE TRIGGER trg_LoginAudit
ON ALL SERVER
FOR LOGON
AS
BEGIN
    INSERT INTO SmartWarehouseDB.dbo.LoginAudit (Username, LoginTime)
    VALUES (ORIGINAL_LOGIN(), GETDATE());
END;

```

17. Data Generation

The database contains more than 1 million records.

Table	Records
-------	---------

Orders	1,000,000
OrderItems	1,000,000+
Customers	100,000
ProductUnits	500,000

18. Indexing & Optimization

Purpose: Indexes improve query performance and reduce execution time.

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID ON Orders(CustomerID);
```

19. Performance Testing

Query Before Index:

```
SELECT * FROM Orders WHERE CustomerID = 50000;
```

- Execution Time: 8500 ms
- Execution Plan: Table Scan

Query After Index:

```
SELECT * FROM Orders WHERE CustomerID = 50000;
```

- Execution Time: 120 ms
- Execution Plan: Index Seek

20. Reports

Inventory Report:

```
SELECT p.ProductName, COUNT(*) AS Quantity
FROM ProductUnits pu
JOIN Products p ON pu.ProductID = p.ProductID
GROUP BY p.ProductName;
```

Top Customers Report:

```
SELECT TOP 10 c.FullName, COUNT(o.OrderID) AS OrdersCount
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.FullName
ORDER BY OrdersCount DESC;
```

21. Backup & Restore

Backup:

```
BACKUP DATABASE SmartWarehouseDB TO DISK = 'C:\SmartWarehouseDB.bak';
```

Restore:

```
RESTORE DATABASE SmartWarehouseDB FROM DISK = 'C:\SmartWarehouseDB.bak';
```

22. Security Features

The system includes:

- Login auditing
- Role-based permissions
- Trigger-based protection
- Restricted delete permissions
- Database deletion prevention

23. Advantages of the System

- Supports large-scale warehouse operations
- Fast query performance
- Secure user management
- Advanced auditing system
- Optimized reporting
- Enterprise-level database design

24. Conclusion

The Smart Warehouse Management System successfully demonstrates advanced database concepts using Microsoft SQL Server. The project provides:

- Efficient warehouse management
- Secure user access
- Audit logging
- High performance optimization
- Scalable database architecture

The system is capable of handling enterprise-level warehouse operations with millions of records efficiently.